Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE
Slot: B

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: Niranjna devi S I **Reg. No: 192519100**

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyze and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localization sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis

- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of k possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for k possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for k = 2, 4, 8, with and without a local sensor), with code and output as proof.

MARK DISTRIBUTION – ASSIGNMENT_1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

Solutions

Task 1 – Design the Agent and Search Algorithms

1.1 Introduction

The proposed system is an autonomous hospital medicine-delivery robot that transports medicines from a central pharmacy to different hospital wards.

The hospital is represented as a graph in which:

- Nodes represent junctions, corridors, lifts and ward entrances.
- Edges represent connections between locations.
- Edge weights represent the travel time required to move between two locations.

The robot uses Artificial Intelligence search algorithms to identify a suitable route from its starting location to the destination.

As the hospital expands, the number of possible states increases from 1,000 to more than 1,000,000. Therefore, efficient search strategies are required to reduce execution time and memory consumption.

1.2 PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

Component	Description
Performance Measure	Minimum travel time, correct medicine delivery, fewer movements, safety and efficient memory usage.
Environment	Hospital corridors, junctions, lifts, pharmacy and patient wards.
Actuators	Wheels, motors, steering mechanism, robotic arm and display system.
Sensors	Camera, LiDAR, ultrasonic sensors, RFID and location sensors.

1.3 Classification of the Environment

Property	Classification	Justification
Observability	Fully observable (initial assumption)	The robot knows its current location and hospital map.
Determinism	Deterministic	Each action produces a predictable movement between connected locations.
Nature	Sequential	Every movement affects the next decision.
Environment	Dynamic	People, hospital staff and obstacles may change corridor conditions.
State space	Discrete	Locations and connections are represented as graph nodes and edges.
Agents	Multi-agent	The robot operates alongside patients, doctors, nurses and other robots.

1.4 View of Artificial Intelligence

The robot follows the Acting Rationally approach.

A rational agent selects actions that produce the best expected outcome based on its available information.

In this application, the robot selects a suitable route to deliver medicines safely while minimizing travel time and unnecessary movements.

This problem is related to early AI research on problem-solving and state-space search, where researchers investigated how machines could solve problems by representing states, actions and goals.

1.5 Problem Formulation

Element	Description
States	All possible hospital locations represented by graph nodes.
Initial State	Central pharmacy or the robot's current location.
Actions	Move to an adjacent connected location.
Transition Model	Moving from one node to another through a valid edge.
Goal Test	Checks whether the robot has reached the requested ward.
Path Cost	Sum of the travel-time costs of all edges in the selected route.

1.6 Proposed Uninformed Search Strategies

We select three meaningfully different search strategies:

1. Breadth-First Search (BFS) – iterative, queue-based.
2. Depth-First Search (DFS) – recursive, stack-based through recursion.
3. Uniform-Cost Search (UCS) – iterative, priority-queue-based.

These strategies are suitable for comparison because they explore the graph differently and have different time, memory and solution-quality characteristics.

Algorithm 1: Breadth-First Search (BFS)

Basic idea:

Breadth-First Search explores the hospital graph level by level. It first visits all locations one movement away from the starting point, followed by locations two movements away.

It uses a FIFO (First In, First Out) queue to manage unexplored nodes.

Pseudocode:

BFS (Graph, Start, Goal):

Create an empty queue Q
Create an explored set
Insert Start into Q
Mark Start as explored

While Q is not empty:

Current = Remove first element from Q

If Current == Goal:
Return the route

For each Neighbour of Current:

If Neighbour is not explored:

Mark Neighbour as explored
Store its parent
Insert Neighbour into Q

Return FAILURE

Advantages:

- Simple to implement.
- Complete for finite graphs.
- Finds the path with the minimum number of edges.

Limitations:

- Requires considerable memory for large graphs.
- Does not guarantee minimum travel time when edge costs differ.

Reason for selection: BFS provides a useful baseline for comparing search performance and is suitable when all movements have equal costs.

Algorithm 2: Depth-First Search (DFS)

Recursive approach

Basic idea:

Depth-First Search explores one path as deeply as possible before returning to explore alternative paths.

It uses recursion to explore connected locations.

Pseudocode:

```
DFS (Graph, Current, Goal, Explored):

    If Current == Goal:
        Return SUCCESS

    Add Current to Explored

    For each Neighbour of Current:

        If Neighbour is not in Explored:

            Store Neighbour's parent
```

If DFS(Graph, Neighbor, Goal, Explored):

Return SUCCESS

Return FAILURE

Advantages:

- Requires less memory than BFS in many graph structures.
- Can reach deep destinations without storing an entire breadth level.

Limitations:

- May explore unnecessary paths.
- Does not guarantee the shortest path.
- Recursive implementation may cause stack overflow for very deep graphs.

Reason for selection: DFS provides a different exploration strategy and helps evaluate how depth-first exploration performs as the hospital graph grows.

Algorithm 3: Uniform-Cost Search (UCS)

Iterative approach

Basic idea:

Uniform-Cost Search expands the location having the lowest accumulated travel cost.

It uses a priority queue rather than an ordinary FIFO queue.

Unlike BFS, UCS considers the actual travel-time cost of every connection.

Pseudocode:

UCS(Graph, Start, Goal):

Create a priority queue PQ

Insert (0, Start) into PQ

Set cost[Start] = 0

While PQ is not empty:

Remove the node with minimum cost

If Current == Goal:
Return the route

For each Neighbor of Current:

NewCost = cost[Current] + EdgeCost

If Neighbor is not visited
OR NewCost < cost[Neighbor]:

Update cost[Neighbor]
Store parent[Neighbor]
Insert (NewCost, Neighbor) into PQ

Return FAILURE

Advantages:

- Considers actual travel time.
- Finds the minimum-cost route when edge costs are non-negative.
- Suitable for hospital environments where different corridors have different travel times.

Limitations:

- Priority-queue operations require additional processing.
- Can consume significant memory when many routes have similar costs.

Reason for selection: UCS is particularly appropriate for the medicine-delivery application because the shortest route in terms of distance or number of movements may not be the fastest route.

1.7 Preprocessing Requirements

Before executing the search algorithms, the hospital map must be converted into an efficient graph representation.

1. Construct an adjacency list containing all connected locations.
2. Store the travel-time cost associated with every edge.
3. Assign unique identifiers to each location.
4. Prepare a visited/explored set to prevent repeated exploration.
5. Maintain a parent dictionary to reconstruct the final route.
6. For UCS, prepare a priority queue to select the lowest-cost location.

Conclusion:

BFS, DFS and UCS provide three different uninformed search strategies. Their performance will be compared based on execution time, number of nodes expanded, memory consumption and the quality of the resulting route.

Task 2 – Identify the Basic Operation:

The basic operation is the operation that contributes most significantly to the execution time of a search algorithm.

Algorithm	Basic operation	Where it occurs
BFS	Node expansion and successor generation	When removing a node from the queue and examining its neighbours.
DFS	Recursive node expansion	When the algorithm visits a neighbour and makes a recursive call.
UCS	Priority-queue removal and relaxation	When extracting the lowest-cost node and updating neighbouring costs.

Explanation

1. **BFS:** The main operation is node expansion. Every expanded node generates its neighbouring locations. As the branching factor increases, more nodes are inserted into the queue, increasing execution time.

2**. DFS:** The main operation is recursive exploration. The algorithm visits a node and recursively explores its successors. If the goal is deep in the graph, a large number of recursive calls may be required.

3. **UCS:** The main operation is extracting the minimum-cost node from the priority queue and relaxing its outgoing edges. As the number of locations and connections increases, priority-queue operations become more frequent.

Conclusion: The number of node expansions and successor examinations determines the search workload, while UCS additionally incurs priority-queue processing overhead.

Task 3 – Derive Time and Space Complexity Mathematically

3.1 Input Parameters

Let:

Symbol	Meaning
(b)	Branching factor (average successors per node)
(d)	Depth of the shallowest goal
(m)	Maximum search depth
(C^*)	Optimal solution path cost
(epsilon)	Minimum positive step cost
(V)	Number of vertices (hospital locations)
(E)	Number of edges (connections)

The total number of nodes in a complete search tree up to depth (d) is:

$$N = 1 + b + b^2 + b^3 + \ldots\ldots + b^d$$

Using the geometric progression formula:

$$N = \frac{b^{d+1} - 1}{b - 1}$$

For large (b) and (d), the highest-level terms dominate the total.

3.2 Breadth-First Search

Time complexity derivation

BFS explores every node from depth 0 to depth \(d\), before finding the shallowest goal.

The number of expanded nodes is approximately:

$$T(b,d) = 1 + b + b^2 + \ldots + b^d$$

Therefore:

$$T(b, d) = \frac{b^{d+1} - 1}{b - 1}$$

for large b:

$$T(b , d) = O(b^{d+1})$$

Space complexity derivation

BFS stores unexplored nodes in its queue. At the deepest level, the frontier can contain approximately (b^d) nodes.

$$S(b , d) = O(b^{d+1})$$

The extra factor accounts for nodes retained at adjacent levels and frontier bookkeeping.

Task 4 – Best, Average and Worst-Case Behaviour

4.1 Breadth-First Search

- Best case: The goal is the starting location. Only one node is examined.
- Average case: The goal is located at an intermediate depth, and BFS explores several nodes at the preceding levels.
- Worst case: The goal is at the deepest level or is unreachable. BFS explores almost the entire reachable search space.

4.2 Depth-First Search

- Best case: The goal is located along the first branch explored. The algorithm reaches it without exploring alternative branches.
- Average case: The goal is located in a branch that requires some backtracking.
- Worst case: The goal is in the last branch explored or is unreachable. DFS may explore almost all nodes up to the depth limit.

4.3 Uniform-Cost Search

- Best case: The goal is the starting location, so the initial cost of zero satisfies the goal test.
- Average case: The goal is reached after exploring several lower-cost routes.
- Worst case: Many low-cost alternative routes exist, causing UCS to expand a large number of nodes before reaching the goal.

Comparison

The number of generated nodes depends on the location of the goal and the structure of the hospital graph.

BFS is affected mainly by goal depth, DFS by the order of branches and maximum depth, and UCS by accumulated path costs.

Therefore, the same hospital graph can produce different execution times for each algorithm.

Task 5 – Perform Asymptotic Analysis

Asymptotic analysis describes how the work and memory requirements grow as the hospital graph becomes larger. Let V be the number of locations, E the number of connections, b the branching factor, d the shallowest goal depth and m the maximum depth. The graph is stored as an adjacency list, which requires $\Theta(V + E)$ preprocessing time and space.

5.1 Growth of BFS, DFS and UCS

Breadth-First Search (BFS): In a tree-search view, BFS may generate $1 + b + b^2 + \ldots + b^d$ nodes, which is $\Theta(b^d)$ for $b > 1$. Thus its time and frontier memory are $O(b^d)$. In a graph implementation with a visited set and adjacency lists, each vertex and edge is processed at most a constant number of times, giving $O(V + E)$ graph-search time and $O(V)$ auxiliary memory. BFS finds the fewest-edge route, but not necessarily the minimum-time route when edge costs differ.

Depth-First Search (DFS): With a depth limit m, the tree-search bound is $O(b^m)$ time and $O(bm)$ auxiliary memory for recursive path exploration. In a finite graph using an explored set, each reachable vertex and edge can be processed once, so the graph-search bound is $O(V + E)$ time and $O(V)$ memory for the explored set, parent information and recursion/stack. DFS does not guarantee a shortest or minimum-cost route.

Uniform-Cost Search (UCS): UCS expands nodes in increasing accumulated path cost. With a binary heap and non-negative edge costs, a common graph-search bound is $O((V + E) \log V)$ time and $O(V + E)$ total storage for the graph, priority queue, costs and parents. Its search memory can be large, but it returns a minimum-cost route when the goal is removed from the priority queue and all edge costs are non-negative.

5.2 Meaning for the hospital

The adjacency list itself grows linearly with the map size, $\Theta(V + E)$. When the hospital grows from 1,000 to 1,000,000 locations, a graph-search algorithm that visits each location once may need roughly one thousand times as many vertex records. However, a tree-search implementation can grow exponentially with depth and become impractical much earlier. BFS often reaches the memory limit first because it stores a broad frontier. DFS uses a smaller active path but can spend a long time in an unhelpful branch. UCS adds priority-queue work

but is the most suitable of these three when travel times differ and the least-cost route matters.

The notation O gives an upper growth bound, Ω gives a lower bound and Θ gives a tight bound when matching upper and lower bounds apply. For a complete search tree explored to depth d, BFS has $\Theta(b^d)$ work in the usual worst-level case. For a finite connected graph traversal, BFS and DFS have $\Theta(V + E)$ work when all vertices and edges are visited. UCS has heap-related logarithmic overhead in its general graph bound. These bounds describe growth, not exact seconds; graph structure and implementation affect measured time.

Task 6 – Formulate the Recurrence Relation

The recursive algorithm selected in Task 1 is Depth-First Search (DFS). Let T(r) be the maximum work of a recursive DFS call when r levels of depth remain in a tree-like search space. Each call checks the goal and marks the current node, then may inspect up to b successors. The non-recursive work per call is $\Theta(b)$, including checking successors and maintaining visited/parent information.

For a worst-case tree in which the goal is not found until all branches have been explored, the recurrence is:

$T(r) = bT(r - 1) + \Theta(b)$, for $r > 0$

$T(0) = \Theta(1)$

Here, the original problem has remaining depth r; each recursive subproblem has remaining depth $r - 1$; and there are up to b recursive calls. The base case occurs when the depth limit is zero, or when a goal is found. If the goal is found early, the algorithm may stop before making all recursive calls. For graph search with an explored set, repeated states are skipped, and the total work is bounded by $O(V + E)$ rather than the full tree bound.

Task 7 – Solve the Recurrence Relation

The substitution/expansion method is applicable because the recurrence expresses a call in terms of smaller calls of the same form. Expanding the recurrence gives:

$T(r) = bT(r - 1) + cb$

$= b[bT(r - 2) + cb] + cb = b^2T(r - 2) + cb^2 + cb$

$= b^rT(0) + cb(1 + b + b^2 + \ldots + b^{r-1})$

Using the geometric-series identity $1 + b + \ldots + b^{r-1} = (b^r - 1)/(b - 1)$, for $b > 1$:

$T(r) = b^r T(0) + cb(b^r - 1)/(b - 1) = \Theta(b^r)$

Therefore, recursive DFS can require exponential time in the remaining depth in the worst-case tree-search model. Its active recursion path uses O(r) stack space, excluding the explored set; graph-search DFS also stores visited nodes and may require O(V) auxiliary memory. The result explains why branch order and depth matter. Iterative deepening repeats depth-limited DFS with limits 0, 1, …, d. Its work is $T(0) + T(1) + \ldots + T(d) = \Theta(b^d)$ for $b > 1$, because the deepest level dominates the sum. It uses O(bd) memory and finds the shallowest goal when all step costs are equal.

Task 8 – Implement and Test the Algorithms

The following Python implementation uses the same adjacency-list graph for BFS, recursive DFS and UCS. It records generated/expanded nodes and maximum frontier size. The fixed seed makes graph generation reproducible. For very large tests, recursive DFS can exceed Python's recursion limit; an iterative DFS stack is a practical engineering modification, but the recursive version below is retained to match the strategy analysed in Tasks 1 and 6.

```
import random, heapq, time
from collections import deque


def make_graph(n, seed=1741):
    rng = random.Random(seed)
    g = [[] for _ in range(n)]
    # A connected backbone plus reproducible extra links
    for u in range(n - 1):
        w = rng.randint(1, 10)
        g[u].append((u + 1, w)); g[u + 1].append((u, w))
    for _ in range(n):
        u, v = rng.randrange(n), rng.randrange(n)
        if u != v:
            w = rng.randint(1, 10)
            g[u].append((v, w)); g[v].append((u, w))
    return g


def bfs(g, start, goal):
```

```
    q = deque([start]); parent = {start: None}; generated = 1
    expanded = 0; max_frontier = 1
    while q:
        u = q.popleft(); expanded += 1
        if u == goal: break
        for v, _ in g[u]:
            if v not in parent:
                parent[v] = u; q.append(v); generated += 1
        max_frontier = max(max_frontier, len(q))
    return result(parent, start, goal), generated, expanded, max_frontier

def dfs_recursive(g, start, goal):
    parent = {start: None}; expanded = 0; generated = 1
    def visit(u):
        nonlocal expanded, generated
        expanded += 1
        if u == goal: return True
        for v, _ in g[u]:
            if v not in parent:
                parent[v] = u; generated += 1
                if visit(v): return True
        return False
    visit(start)
    return result(parent, start, goal), generated, expanded, len(parent)

def ucs(g, start, goal):
    pq = [(0, start)]; dist = {start: 0}; parent = {start: None}
    generated = 1; expanded = 0; max_frontier = 1
    while pq:
        cost, u = heapq.heappop(pq)
        if cost != dist.get(u): continue
        expanded += 1
        if u == goal: break
        for v, w in g[u]:
            nc = cost + w
            if nc < dist.get(v, float("inf")):
                dist[v] = nc; parent[v] = u
```

```
                heapq.heappush(pq, (nc, v)); generated += 1
            max_frontier = max(max_frontier, len(pq))
        return result(parent, start, goal), generated, expanded, max_frontier


def result(parent, start, goal):
    if goal not in parent: return None
    path = []; u = goal
    while u is not None: path.append(u); u = parent[u]
    return list(reversed(path))


if __name__ == "__main__":
    for n in (1000, 10000):  # extend to 100000 and 1000000 on a suitable machine
        g = make_graph(n, seed=1741)
        for name, fn in (("BFS", bfs), ("DFS", dfs_recursive), ("UCS", ucs)):
            t0 = time.perf_counter(); out = fn(g, 0, n - 1)
            elapsed = time.perf_counter() - t0
            path, generated, expanded, frontier = out
            print(n, name, bool(path), len(path)-1 if path else None,
                  generated, expanded, frontier, round(elapsed, 4))
```

Test cases and expected behaviour:

(1) start = goal: every algorithm should return a one-node route; (2) reachable goal: the connected backbone guarantees a route from node 0 to node n−1; (3) unreachable goal: remove all edges to a chosen isolated node and use it as the goal, which should return failure. BFS and DFS return a route but do not guarantee minimum travel cost. UCS returns a minimum-cost route for non-negative weights.

The code above is reproducible and prints actual values on execution. The 100,000- and 1,000,000-state runs should be executed on the target computer and the printed measurements pasted into Task 9; runtime values are machine-dependent, so they should not be invented in a report. The recursive DFS implementation may need to be replaced with an explicit stack for deep/large test graphs.

Task 9 – Record Experimental Performance

Performance should be measured on identical graphs and start/goal pairs for all algorithms. Use seed 1741 and run each configuration at least five times.

Record the mean elapsed time, and record generated nodes, expanded nodes, maximum frontier size, route depth and path cost. The maximum frontier is a memory proxy, not a direct measurement of RAM.

9.1 Results table template

State count | Algorithm | Generated | Expanded | Max frontier | Route depth | Path cost | Mean time (s)

1,000 | BFS / DFS / UCS | record from program output

10,000 | BFS / DFS / UCS | record from program output

100,000 | BFS / DFS / UCS | record from program output

1,000,000 | BFS / DFS / UCS | record from program output

For each run, use time.perf_counter() immediately before and after the algorithm. Repeat five times and calculate mean time = (t1 + t2 + t3 + t4 + t5) / 5. For path cost, sum the edge weights along the returned path. For BFS and DFS, label optimality as “not guaranteed”; for UCS, label it “optimal” when all edge costs are non-negative and the implementation terminates when the goal is removed at its current best cost.

Suggested graphs: plot state count on the horizontal axis and mean execution time on the vertical axis; use a logarithmic vertical axis if the times vary widely. Create a second graph for maximum frontier size. Keep the seed and test conditions in the caption so the experiment can be reproduced.

Task 10 – Compare Theory with Experimental Behaviour

The theoretical and experimental results should be compared using the same graph representation and input pairs. For graph-search BFS and DFS with duplicate detection, the number of expanded nodes should generally increase in proportion to the reachable vertices and edges when most of the graph is visited. If the goal is near the start, the algorithms may stop early and the measured work will be much smaller than the worst-case bound.

BFS usually maintains a broad queue, so its maximum frontier can increase sharply when many nodes occur at the same depth. DFS often has a smaller active path, although the explored set and parent dictionary still grow with visited locations. UCS performs priority-queue insertions and removals and may

update a location more than once; its elapsed time can therefore be higher than BFS/DFS even when it finds a better travel-time route.

Differences between theory and measurements may result from the graph's degree distribution, the goal location, duplicate edges, priority-queue updates, Python object overhead, operating-system scheduling and available RAM. A graph with a direct route to the goal can make every algorithm look fast; an unreachable goal forces a complete search of the reachable component. Therefore, no single timing result should be treated as universal. Use repeated runs and report averages.

Task 11 – Recommend and Defend the Final Design

For the original hospital task, Uniform-Cost Search (UCS) is recommended as the route-planning baseline because each corridor connection has a travel-time cost and the robot should minimize total delivery time rather than merely the number of movements. UCS is complete and optimal for a finite graph with non-negative edge costs, provided it uses correct priority-queue handling and tests the goal when the minimum-cost entry is removed.

BFS is simpler and can be faster when all connections have equal costs, but it can select a slower route when travel times differ. DFS may use a small active search path, but its route depends on branch order and is not guaranteed to be shortest or least-cost. UCS has additional heap-processing and memory costs, so the measured results from Task 9 should be used to confirm that the target computer can handle the hospital map.

Preprocessing the adjacency list costs $\Theta(V + E)$ time and space and can be reused for repeated deliveries while the map remains unchanged. For a hospital with more than one million locations, the implementation should use compact graph storage, avoid duplicate edges, monitor memory and consider route-planning methods with admissible heuristics or hierarchical maps if the assignment permits informed search. Under the required uninformed strategies, UCS is the strongest choice when minimum travel time is the main performance measure.

Task 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

12.A Impact Analysis

The original problem assumes that the robot knows its exact current location. After a restart or localization-sensor fault, it knows only that it is at one of k possible junctions. The environment is therefore partially observable, and the start state is no longer a single graph node. The robot must represent its knowledge as a belief state: a set of locations where it may currently be.

The new state includes the set of possible locations because two robots with the same physical map but different location uncertainty require different decisions. The goal condition is also stronger: the chosen plan must reach the pharmacy docking station regardless of which candidate location is the true one.

12.B Evaluation of the Original Design

BFS, DFS and UCS can still search a graph, but using one guessed starting node is unsafe because the robot could follow a route that is invalid or ineffective from its actual location. A belief-state search instead treats each subset of locations as a state. For k candidate locations, there can be up to 2^k possible subsets, including the empty set; if the belief is guaranteed non-empty, there are $2^k - 1$ non-empty subsets.

For k = 2, 4 and 8, the maximum number of non-empty belief states is respectively 3, 15 and 255. This grows exponentially in k. Therefore, applying the original search unchanged is not acceptable when localization is uncertain, particularly if the candidate set becomes large.

12.C Adaptation or Redesign

The design is modified to use belief-state planning with local sensing. First, the robot stores the candidate set $B = \{v1, v2, \ldots, vk\}$. It then chooses a safe action that is valid for every location in B, predicts the possible next locations, and uses a local sensor to remove candidates inconsistent with the observation. Search continues over the updated belief set until the belief is the docking location or the chosen conditional plan guarantees docking from every remaining candidate.

Without sensing, the planner may need to find a common action sequence that succeeds for every possible start, which can be impossible in some maps. With a reliable local sensor, the belief can shrink after each observation, reducing uncertainty and often reducing the search effort. The worst-case belief-state space remains O(2^V) for a full graph with V locations, although this experiment only considers a small candidate set k. The local sensor adds observation branches but can greatly reduce the number of active candidates.

12.D Evidence-Based Defence and Small Experiment

For a simple count-based experiment, assume every non-empty subset of the k candidate junctions is a possible belief state. The following short program reports the theoretical belief-state count and the number of candidates remaining after a perfect local sensor identifies the actual junction.

```
def belief_experiment(k_values=(2, 4, 8)):
    for k in k_values:
        without_sensor = 2**k - 1
        with_sensor = k  # one singleton belief per possible observation
        print(f"k={k}: without_sensor={without_sensor}, with_sensor={with_sensor}, initial_candidates={k},
after_perfect_sensor=1")


            f" after_perfect_sensor=1")


belief_experiment()
```

Expected output:

```
k=2: without_sensor=3, with_sensor=2, initial_candidates=2, after_perfect_sensor=1
k=4: without_sensor=15, with_sensor=4, initial_candidates=4, after_perfect_sensor=1
k=8: without_sensor=255, with_sensor=8, initial_candidates=8, after_perfect_sensor=1
```

Interpretation: the no-sensor belief-state space grows from 3 to 255 as k rises from 2 to 8. With a perfect local sensor, there are k possible singleton observations, and after the observation the robot knows its exact junction (one candidate remains). This experiment measures belief-state counts, not physical route length. Actual plan length depends on the hospital graph, the available actions and sensor accuracy; it must be measured on a specified graph. The design should therefore be modified rather than retained unchanged: use belief-state reasoning and local sensing, and replan after each observation. If the

sensor is unreliable, maintain a probability distribution or a set of possible states instead of assuming perfect localization.

Overall Conclusion

BFS, DFS and UCS provide different trade-offs for hospital route planning. Asymptotic analysis explains their growth, while reproducible experiments are needed to compare actual runtime and memory behaviour. UCS is recommended for minimizing travel time under the original assumptions. When localization becomes uncertain, the robot must represent a set of possible locations and combine planning with sensing.

****